

山东大学 网络空间安全 学院

网络安全创新创业实践 课程实验报告

小组成员：张涵/彭炜霖/李庭跃

实验题目：任务 1：比特币测试网交易发送与底层数据解析

实验学时：4h

实验日期：2026 年 7 月 20 日

实验目的：

- (1) 掌握比特币测试网的基本交互操作，包括轻节点钱包的配置、测试币水龙头的使用及交易发送。
- (2) 通过编写 Python 数据解析脚本，亲自动手复现并验证工作量证明 (PoW) 机制中的双重 SHA-256 哈希计算过程。

实验步骤：

一 实验环境：

1.1 软件环境：

- Electrum 4.8.0 轻量级钱包 (Testnet 模式)
- mempool.space 比特币测试网区块链浏览器
- Python 3

1.2 硬件环境：

- 个人计算机 (Ubuntu 虚拟机终端环境)

二 实验内容

本次实验要求完成从链上交互到本地数据逆向解析的完整闭环，主要包含以下内容：

- (1) 网络交互与交易上链：在 Linux 终端下载并运行 Electrum 钱包，申请测试币并对外发送一笔转账。
- (2) 原始数据获取：利用 Mempool 浏览器查询所发交易的 TXID，分别提取出该笔交易的完整 Raw Hex，以及包含该交易的区块其标准的 80 字节区块头原始十六进制数据。
- (3) 逐字节数据解析：根据区块链协议规范，编写解析脚本。最终实现对输入输出金额、脚本长度及锁定时间的精准拆解，并成功验证 PoW 工作量哈希。

代码实现：

一 Python 解析脚本 (task1.py)

为了兼容隔离见证交易，并严格校验区块头长度以防数据抓取错误，本实验设计了如下解析代码：

```
1 import hashlib
2
3 def little_endian_to_int(hex_str):
4     #将小端序十六进制字符串转换为整数
5     return int.from_bytes(bytes.fromhex(hex_str), byteorder='little')
6
7 def double_sha256(hex_string):
8     #计算双重SHA256
9     first_hash = hashlib.sha256(bytes.fromhex(hex_string)).digest()
10    second_hash = hashlib.sha256(first_hash).digest()
11    return second_hash[::-1].hex()
12
13 def parse_block_header(header_hex):
14
15     print(" 解析 Block Header ")
16     header_hex = header_hex.strip()
17
18     version = header_hex[0:8]
19     prev_hash = header_hex[8:72]
20     merkle_root = header_hex[72:136]
21     timestamp = header_hex[136:144]
22     bits = header_hex[144:152]
23     nonce = header_hex[152:160]
24
25     print(f"1. Version (版本号): {version} ->
26           {little_endian_to_int(version)}")
27     print(f"2. Previous Block Hash: \n {prev_hash}")
28     print(f"3. Merkle Root: \n {merkle_root}")
29     print(f"4. Timestamp (时间戳): {timestamp} ->
30           {little_endian_to_int(timestamp)}")
31     print(f"5. Difficulty Target (Bits): {bits}")
32     print(f"6. Nonce (随机数): {nonce} -> {little_endian_to_int(nonce)}")
33
34     calculated_hash = double_sha256(header_hex)
35     print(f"\n[PoW 验证] 计算本区块头的双重 SHA256 哈希:\n{calculated_hash}")
36
37 def parse_transaction(raw_tx):
38     print(" 解析 Transaction ")
```

```

37
38 raw_tx = raw_tx.strip()
39 cursor = 8
40 version = raw_tx[:8]
41 print(f"Version: {version} -> {little_endian_to_int(version)}")
42
43 if raw_tx[cursor:cursor + 4] == "0001":
44     cursor += 4
45
46 in_count_hex = raw_tx[cursor:cursor + 2]
47 in_count = little_endian_to_int(in_count_hex)
48 print(f"\nInput Count (输入数量): {in_count_hex} -> {in_count}")
49 cursor += 2
50
51 for i in range(in_count):
52     print(f"\n--- [Input {i}] ---")
53     txid_hex = raw_tx[cursor:cursor + 64]
54     print(f"Previous TX Hash: \n
55           {bytes.fromhex(txid_hex)[::-1].hex()}")
56     cursor += 64
57
58     vout = raw_tx[cursor:cursor + 8]
59     print(f"Previous Output Index: {vout} ->
60           {little_endian_to_int(vout)}")
61     cursor += 8
62
63     script_sig_len = little_endian_to_int(raw_tx[cursor:cursor + 2])
64     cursor += 2
65     print(f"Unlocking Script Length: {script_sig_len} 字节")
66
67     if script_sig_len > 0:
68         script_sig = raw_tx[cursor:cursor + (script_sig_len * 2)]
69         print(f"Unlocking Script: {script_sig}")
70         cursor += (script_sig_len * 2)
71     else:
72         print(f"Unlocking Script: 空 ")
73
74     sequence = raw_tx[cursor:cursor + 8]
75     print(f"Sequence Number: {sequence}")
76     cursor += 8
77
78 out_count_hex = raw_tx[cursor:cursor + 2]
79 out_count = little_endian_to_int(out_count_hex)

```

```

78 print(f"\nOutput Count (输出数量): {out_count_hex} -> {out_count}")
79 cursor += 2
80
81 for i in range(out_count):
82     print(f"\n--- [Output {i}] ---")
83     amount = raw_tx[cursor:cursor + 16]
84     satoshi = little_endian_to_int(amount)
85     print(f"Amount (聪/Satoshi): {amount} -> {satoshi} ({satoshi /
86         1e8} tBTC)")
87     cursor += 16
88
89     script_pubkey_len = little_endian_to_int(raw_tx[cursor:cursor + 2])
90     cursor += 2
91     print(f"Locking Script Length: {script_pubkey_len} 字节")
92
93     script_pubkey = raw_tx[cursor:cursor + (script_pubkey_len * 2)]
94     print(f"Locking Script: \n {script_pubkey}")
95     cursor += (script_pubkey_len * 2)
96
97 locktime = raw_tx[-8:]
98 print(f"\nLocktime (锁定时间): {locktime} ->
99     {little_endian_to_int(locktime)}")
100
101 tx_hex = (
102     "02000000000101bfa57dc098d76c993f66084b5bcbefd25f"
103     "d185f65cf07f2fa2edf58dd3cdff5f000000000fdffffff"
104     "021027000000000000160014c8c43f9b09e2aadeb3fc1d20"
105     "0da042443bfd3b90a47d010000000001600148720f7d193"
106     "ac7dacf3ccee15d44a5a1b970aac290247304402203b2be4"
107     "07b36e2e30da0e4910d59ba4fca11773762a470a3d9726f0"
108     "ce43470426022016f44f3b79a993556657d31f3bd15e2cad"
109     "3bb56699e6579e6dc82652b4aecb47012102fbc4a496077b"
110     "6a3368989f25e72e60b77270b36642689dbca75f731a693f"
111     "4b1d00000000"
112 )
113
114 header_hex = (
115     "04000020aa40636186cdd62d715c429ca2d7e3b7b09056f7"
116     "00f5f72dec254b3300000000c80a53071babea49096231d0"
117     "76ce91fc8509f15c99b56dc6f7de05e0feb794118adb5d6a"
118     "ffff001d59f13a5f"
119 )

```

```

119
120 if __name__ == "__main__":
121     parse_block_header(header_hex)
122     parse_transaction(tx_hex)

```

结果输出分析：

一 终端执行结果

通过执行 ‘task1.py’ 脚本，针对抓取到的测试网数据进行解析，控制台成功输出了符合逻辑的区块结构与交易明细：

```

1      解析 Block Header
2  1. Version (版本号): 04000020 -> 536870916
3  2. Previous Block Hash:
4      aa40636186cdd62d715c429ca2d7e3b7b09056f700f5f72dec254b3300000000
5  3. Merkle Root:
6      c80a53071babea49096231d076ce91fc8509f15c99b56dc6f7de05e0feb79411
7  4. Timestamp (时间戳): 8adb5d6a -> 1784535946
8  5. Difficulty Target (Bits): ffff001d
9  6. Nonce (随机数): 59f13a5f -> 1597698393
10
11 [PoW 验证] 计算本区块头的双重 SHA256 哈希:
12 000000001f28dc7c06309babbc26e9f461a58fc8f4f072b9835366b58fcae6af
13      解析 Transaction
14 Version: 02000000 -> 2
15
16 Input Count (输入数量): 01 -> 1
17
18 --- [Input 0] ---
19 Previous TX Hash:
20 5fffcdd38df5eda22f7ff05cf685d15fd2efcb5b4b08663f996cd798c07da5bf
21 Previous Output Index: 00000000 -> 0
22 Unlocking Script Length: 0 字节
23 Unlocking Script: 空
24 Sequence Number: fdffffff
25
26 Output Count (输出数量): 02 -> 2
27
28 --- [Output 0] ---
29 Amount (聪/Satoshi): 1027000000000000 -> 10000 (0.0001 tBTC)
30 Locking Script Length: 22 字节
31 Locking Script:

```

```

32 0014c8c43f9b09e2aadeb3fc1d200da042443bfd3b90
33
34 --- [Output 1] ---
35 Amount (聪/Satoshi): a47d010000000000 -> 97700 (0.000977 tBTC)
36 Locking Script Length: 22 字节
37 Locking Script:
38 00148720f7d193ac7dacf3ccee15d44a5a1b970aac29
39
40 Locktime (锁定时间): 00000000 -> 0

```

测试结果截图如下：



图 1：实验结果截图